



Universidad
Carlos III de Madrid

Universidad Carlos III
Curso Procesadores del Lenguaje 25/26-2C
Analizador Semántico

MEMORIA P3

Grupo: **80**

Autores:

Álvaro García González - 100522223@alumnos.uc3m.es
Samuel Gómez Fernández - 100522224@alumnos.uc3m.es

Índice

1. Introducción.....	3
1.1. Parte Opcional: Generación de Código Intermedio.....	3
2. Arquitectura de la Solución.....	3
2.1. Descripción General de parser.py.....	3
2.2. Descripción General de main.py.....	4
2.3. Variables e Indicadores de Estado.....	4
3. Diseño de la Solución.....	5
3.1. Definición Formal de la Gramática.....	5
3.2. Estructuras Semánticas y Gestión de Ámbitos.....	8
3.3. Representación Interna de Expresiones.....	9
3.4. Métodos Auxiliares del parser.....	9
3.4.1. Gestión de la Tabla de Símbolos.....	9
3.4.2. Comprobación y Conversión de Tipos.....	9
3.4.3. Validación Semántica.....	10
3.4.4. Exportación de Tablas Semánticas.....	10
3.4.5. Utilidades Específicas de cuartetos.....	11
3.5. Comprobaciones Semánticas Implementadas.....	11
3.5.1. Declaraciones, Asignaciones y Sentencias Simples.....	11
3.5.2. Expresiones.....	12
3.5.2.1. Operaciones aritméticas.....	12
3.5.2.2. Operaciones Comparativas.....	12
3.5.2.3. Operaciones Lógicas.....	13
3.5.2.4. Operadores Unarios.....	13
3.5.3. Registros.....	13
3.5.4. Funciones.....	14
3.5.4.1. Declaración y Sobrecarga.....	14
3.5.4.2. Invocación y Resolución de Sobrecarga.....	15
3.5.4.3. Validación de return.....	15
3.5.5. Control de Flujo.....	16
3.6. Gestión de Errores Semánticos.....	16
3.7. Diseño Parte Opcional.....	17
4. Batería de Pruebas.....	18
4.1. Tabla de cobertura.....	18
4.2. Análisis de resultados.....	19
4.2.1. Ficheros .symbols.....	19
4.2.2. Ficheros .records.....	20
4.2.3. Ficheros .functions.....	20

4.2.4. Ficheros .quartets.....	20
5. Conclusiones.....	21
ANEXO. Declaración de Uso de Inteligencia Artificial.....	21

1. Introducción

El presente documento constituye la memoria técnica de la tercera fase (P3) del proyecto de compilación de la asignatura. Esta fase final se centra en el diseño, implementación y validación de un **analizador semántico** integrado en el parser del lenguaje de programación Lava, construido sobre el analizador sintáctico y léxico desarrollado en las entregas anteriores y siguiendo estrictamente la especificación del enunciado.

El propósito de la práctica es triple. En primer lugar, **extender la gramática formal ya existente con acciones semánticas** capaces de detectar y reportar errores de tipo, redeclaración de variables, uso incorrecto de sentencias de control y validación de firmas de funciones. En segundo lugar, **implementar las estructuras de datos necesarias para dar soporte a dicho análisis**: una tabla de variables, una tabla de registros, una tabla de funciones y una pila de ámbitos que modela correctamente el alcance léxico del lenguaje, todo ello mediante la librería *ply.yacc*.

1.1. Parte Opcional: Generación de Código Intermedio

En esta entrega se ha implementado adicionalmente la **parte opcional** de la práctica: la **generación de código intermedio en formato de cuartetos** (ficheros *.quartets*). Esta funcionalidad se integró directamente en *parser.py*, aprovechando el flujo semántico ya existente, de modo que cada regla gramatical, una vez validada semánticamente, emite los cuartetos correspondientes.

La implementación de esta fase supuso un reto adicional, dado que la generación de código intermedio no había sido abordada al detalle en clase en el momento de la entrega. Para afrontarlo, se recurrió al aprendizaje autónomo y al uso de herramientas de IA como apoyo formativo, tal y como queda detallado en el **Anexo de Declaración de Uso de Inteligencia Artificial**.

Esta memoria detalla las decisiones de diseño adoptadas, justificando técnicamente cada elección realizada y las conclusiones obtenidas. El objetivo es **proporcionar una documentación clara que facilite la comprensión del código implementado**.

2. Arquitectura de la Solución

2.1. Descripción General de parser.py

parser.py implementa la clase **ParserClass**, que actúa como analizador sintáctico, semántico y generador de código intermedio del lenguaje Lava. Su funcionamiento: recibe el flujo de tokens producido por **LexerClass**, aplica las reglas de producción de la gramática y, de forma simultánea, ejecuta las acciones semánticas asociadas a cada regla. Al finalizar el análisis, exporta los resultados a

tres ficheros de salida *.symbols*, *.records* y *.functions* que reflejan el estado final de las tablas semánticas; y *.quartets* (código intermedio).

2.2. Descripción General de main.py

main.py representa la ejecución del compilador. Tras invocar el parser, comprueba si existen errores sintácticos o semánticos. Si no los hay, exporta las tablas semánticas mediante **exportar_tablas_semanticas(entrada)** y, adicionalmente, exporta el código intermedio mediante **exportar_cuartetos(entrada)**.

2.3. Variables e Indicadores de Estado

El constructor **__init__** inicializa un conjunto de variables que controlan el estado del análisis a lo largo de toda la ejecución del parser:

Estado semántico:

- **has_syntax_error**: indica si se ha producido algún error sintáctico durante el análisis.
- **has_semantic_error**: indica si se ha producido algún error semántico.
- **current_function**: almacena el contexto de la función que se está analizando en cada momento, incluyendo su nombre, tipo de retorno y si ya se ha encontrado un *return* válido.
- **loop_depth**: contador de bucles anidados activos, utilizado para validar el uso correcto de *break*.
- **_has_flow**: marca si el programa contiene estructuras de control o funciones, lo que determina el formato de exportación de la tabla de variables.
- **_decl_type**, **_decl_name**, **_decl_line**: variables auxiliares que almacenan temporalmente los datos de la declaración que se está procesando, necesarias para comunicar información entre reglas de producción no adyacentes.

Estado de generación de cuartetos:

- **self.quartets**: lista de tuplas (*operador*, *arg1*, *arg2*, *resultado*) que actúa como buffer de cuartetos emitidos.
- **self.temp_counter**: contador entero para generar nombres de temporales (*@T1*, *@T2*, ...).
- **self.label_counter**: contador entero para generar nombres de etiquetas (*@L1*, *@L2*, ...).
- **self.if_codegen_stack**: pila de diccionarios con el contexto de cada *if/else* activo (etiquetas *else_label*, *end_label* y flag *has_else*).
- **self.loop_codegen_stack**: pila de diccionarios con el contexto de cada bucle activo (etiquetas *start*, *end* y campo *kind* con valor *while* o *do*).

Todo estos estados se reinician en **_reiniciar_estado_semantico()**, que limpia tanto las tablas semánticas como el estado de generación de cuartetos.

Asimismo, la clase **ParserClass()** define el elemento estático **DEFAULT_TYPES**: diccionario que asocia cada tipo básico del lenguaje con su valor por defecto. Se utiliza para inicializar variables declaradas sin valor explícito.

3. Diseño de la Solución

3.1. Definición Formal de la Gramática

A continuación se presenta la **gramática** completa del lenguaje Lava en notación **BNF**. Debe tenerse en cuenta que algunos no terminales auxiliares como `<if_mark>`, `<while_start>` o `<inicio_funcion_tipada>` no introducen nuevas construcciones del lenguaje, sino que actúan como marcadores internos para ejecutar acciones semánticas y de generación de código intermedio durante el parseo.

```
Shell
// Gramática
programa      ::= elementos_opt

elementos_opt ::= λ | elementos

elementos     ::= elementos elemento | elemento

elemento      ::= SEMICOLON
                | declaracion_funcion_void
                | declaracion_o_funcion_tipada
                | declaracion_record SEMICOLON
                | sentencia_bloque
                | sentencia_simple SEMICOLON

// Declaraciones globales y funciones
declaracion_funcion_void ::= VOID ID LPAREN parametros_opt RPAREN
                           inicio_funcion_void bloque_void fin_funcion_void
declaracion_o_funcion_tipada ::= tipo ID marcar_declaracion_tipada resto_tipado_programa

resto_tipado_programa ::=
    LPAREN parametros_opt RPAREN inicio_funcion_tipada bloque fin_funcion_tipada
    | ASSIGN expresion SEMICOLON
    | resto_lista_ids SEMICOLON

resto_lista_ids ::= λ | resto_lista_ids COMMA ID

// Bloques y control de flujo en funciones void
bloque_void      ::= LBRACE elementos_bloque_void_opt RBRACE

elementos_bloque_void_opt ::= λ | elementos_bloque_void

elementos_bloque_void ::= elementos_bloque_void elemento_bloque_void
                       | elemento_bloque_void

elemento_bloque_void ::= SEMICOLON | sentencia_bloque_void
```

```
| declaracion_variable SEMICOLON
| sentencia_simple_void SEMICOLON

sentencia_bloque_void ::= bloque_void | sentencia_if_void | sentencia_while_void
                        | sentencia_do_while_void

sentencia_if_void ::= IF LPAREN expresion RPAREN
                    if_mark bloque_void else_void_opt if_end_mark

else_void_opt      ::= λ | else_mark ELSE bloque_void

sentencia_while_void ::=
    WHILE while_start LPAREN expresion RPAREN
    while_cond_mark entrar_bucle bloque_void salir_bucle while_end

sentencia_do_while_void ::=
    DO do_start entrar_bucle bloque_void salir_bucle
    WHILE LPAREN expresion RPAREN SEMICOLON do_end

sentencia_simple_void ::= asignacion | print_stmt | BREAK | expresion

// Bloques y control de flujo general
bloque                ::= LBRACE elementos_bloque_opt RBRACE

elementos_bloque_opt ::= λ | elementos_bloque

elementos_bloque ::= elementos_bloque elemento_bloque | elemento_bloque

elemento_bloque ::= SEMICOLON | sentencia_bloque | declaracion_variable SEMICOLON
                 | sentencia_simple SEMICOLON

sentencia_bloque ::= bloque | sentencia_if | sentencia_while | sentencia_do_while
sentencia_if     ::= IF LPAREN expresion RPAREN if_mark bloque else_opt if_end_mark

else_opt          ::= λ | else_mark ELSE bloque
sentencia_while  ::=
    WHILE while_start LPAREN expresion RPAREN
    while_cond_mark entrar_bucle bloque salir_bucle while_end

sentencia_do_while ::=
    DO do_start entrar_bucle bloque salir_bucle
    WHILE LPAREN expresion RPAREN SEMICOLON do_end

// Sentencias simples y declaraciones
sentencia_simple ::= asignacion | print_stmt | BREAK | RETURN expresion | expresion

declaracion_variable ::= tipo lista_ids | tipo ID ASSIGN expresion

lista_ids            ::= ID | lista_ids COMMA ID

asignacion           ::= acceso ASSIGN expresion
```

```
print_stmt      ::= PRINT LPAREN argumentos_opt RPAREN
```

// Registros parámetros y tipos

```
declaracion_record ::= RECORD ID LPAREN campos_record_opt RPAREN
```

```
campos_record_opt ::= λ | campos_record
```

```
campos_record    ::= campos_record COMMA campo_record_item | campo_record_item
```

```
campo_record_item ::= tipo ID | ID
```

```
parametros_opt   ::= λ | parametros
```

```
parametros       ::= parametros COMMA parametro | parametro
```

```
parametro        ::= tipo ID
```

```
tipo             ::= INT | FLOAT | CHAR | BOOLEAN | ID
```

// Expresiones

```
expresion        ::= expresion OR expresion  
                  | expresion AND expresion  
                  | expresion EQUAL expresion  
                  | expresion GREATER expresion  
                  | expresion GREATER_EQUAL expresion  
                  | expresion LESS expresion  
                  | expresion LESS_EQUAL expresion  
                  | expresion PLUS expresion  
                  | expresion MINUS expresion  
                  | expresion TIMES expresion  
                  | expresion DIVIDE expresion  
                  | NOT expresion  
                  | MINUS expresion  
                  | PLUS expresion  
                  | primario
```

```
primario         ::= literal | acceso | llamada | LPAREN expresion RPAREN | instanciacion
```

```
acceso           ::= ID | acceso DOT ID
```

```
llamada          ::= ID LPAREN argumentos_opt RPAREN
```

```
instanciacion    ::= NEW ID LPAREN argumentos_opt RPAREN
```

```
argumentos_opt   ::= λ | argumentos
```

```
argumentos       ::= argumentos COMMA expresion | expresion
```

```
literal          ::= INT_VALUE | FLOAT_VALUE | CHAR_VALUE | TRUE | FALSE
```

```
// Marcadores auxiliares
-- Marcadores de codegen (producen λ):
if_mark      ::= λ
else_mark    ::= λ
if_end_mark  ::= λ
while_start  ::= λ
while_cond_mark ::= λ
while_end    ::= λ
do_start     ::= λ
do_end       ::= λ
entrar_bucle ::= λ
salir_bucle  ::= λ

-- Marcadores semánticos (producen λ):
inicio_funcion_void    ::= λ
fin_funcion_void       ::= λ
inicio_funcion_tipada  ::= λ
fin_funcion_tipada     ::= λ
marcar_declaracion_tipada ::= λ

// Producción vacía
λ      ::= <empty>
```

3.2. Estructuras Semánticas y Gestión de Ámbitos

El análisis semántico se apoya en un conjunto reducido de estructuras de datos inicializadas en el constructor de **ParserClass**. La información semántica principal se organiza en tres diccionarios independientes:

- **self.symbols** (variables globales declaradas): Almacena cada variable como *nombre* → (*tipo*, *valor*).
- **self.records** (tipos compuestos declarados con *record*): Almacena cada registro como *nombre_registro* → {*nombre_campo* → *tipo_campo*}, preservando la estructura de sus campos.
- **self.functions** (funciones declaradas): Almacena cada función como *nombre_función* → [*lista de firmas*], permitiendo la sobrecarga por número o tipo de parámetros.
Además, cada firma se representa mediante un diccionario con dos campos: **params**, que contiene la lista de pares (*tipo*, *nombre*) de los parámetros, y **return_type**, que indica el tipo de retorno de la función, incluyendo el caso especial *void*.

Además de estas tablas, el parser mantiene una **pila de ámbitos** mediante **self.stack**. Esta pila modela el alcance léxico del lenguaje: el primer elemento corresponde siempre al ámbito global, representado por **self.symbols**, y cada vez que se entra en una función se añade un nuevo diccionario local con sus parámetros y variables internas.

Cuando el parser analiza el cuerpo de una función, crea un nuevo ámbito local y lo apila. Al finalizar la función, dicho ámbito se elimina. De este modo, las variables locales tienen prioridad sobre las globales durante la búsqueda de símbolos, pero dejan de existir al salir de la función. Esta estrategia evita colisiones indebidas entre ámbitos y permite comprobar correctamente redeclaraciones, accesos a variables y parámetros de función.

3.3. Representación Interna de Expresiones

Con la introducción de la generación de código intermedio, la representación interna de todas las expresiones pasó de ser una tupla de longitud exacta 2 (*tipo*, *valor*) a una tupla de longitud 3 (*tipo*, *valor*, *ref*):

- ***tipo*** recoge el tipo semántico inferido.
- ***valor*** almacena el valor calculado cuando puede conocerse en tiempo de análisis.
- ***ref*** representa el operando que se usará directamente en los cuartetos (un literal serializado, el nombre de una variable o un temporal *@Tn*).

Para construir y descomponer estas expresiones de forma uniforme se introdujeron dos métodos auxiliares que se explicaran con detalle en el siguiente apartado: ***_expr*** y ***_extraer_expr***.

3.4. Métodos Auxiliares del parser

3.4.1. Gestión de la Tabla de Símbolos

Estos métodos operan directamente sobre la pila de ámbitos (*self.stack*) para localizar y modificar variables ya declaradas:

- ***_buscar_simbolo(name)***: recorre la pila de ámbitos de más interno a más externo y devuelve la entrada correspondiente al nombre buscado, o *None* si no existe. Este orden de recorrido garantiza que las variables locales tienen precedencia sobre las globales.
- ***_actualizar_simbolo(name, value)***: localiza el símbolo en la pila siguiendo el mismo criterio de proximidad y actualiza su valor manteniendo el tipo original. Devuelve *True* si la actualización fue exitosa o *False* si el símbolo no existe.

3.4.2. Comprobación y Conversión de Tipos

Este es el grupo más extenso de métodos auxiliares. Su función es determinar la compatibilidad entre tipos y realizar las conversiones necesarias durante el análisis:

- ***_es_tipo_valido(type_name)***: comprueba si un nombre de tipo corresponde a un tipo básico del lenguaje o a un registro previamente declarado.
- ***_es_tipo_numerico(type_name)***: determina si un tipo es numérico, considerando como tales *char*, *int* y *float*.
- ***_auto_convert(source_type, target_type)***: indica si existe una conversión implícita permitida entre dos tipos. Las conversiones válidas son *char* $\rightarrow$ *int*, *char* $\rightarrow$ *float* e *int* $\rightarrow$ *float*, siguiendo una jerarquía de promoción numérica.

- **_costo_conversion(source_type, target_type)**: asigna un coste numérico a cada conversión posible (*char* → *int*: 1, *int* → *float*: 1, *char* → *float*: 2), utilizado para resolver la sobrecarga de funciones eligiendo la firma con menor coste total de conversión.
- **_tipo_numerico_comun(left_type, right_type)**: determina el tipo resultante de operar dos operandos numéricos, aplicando la regla de promoción: *float* tiene mayor prioridad que *int*, e *int* mayor que *char*.
- **_convertir_numero(value, source_type, target_type)**: convierte un valor numérico concreto al tipo objetivo. Gestiona casos especiales como el *char* vacío (tratado como cero) y la conversión de caracteres mediante *ord()*.
- **_normalizar_operandos_numericos(left_type, left_value, right_type, right_value)**: combina **_tipo_numerico_comun** y **_convertir_numero** para llevar ambos operandos de una expresión binaria al mismo tipo antes de evaluarla.
- **_convertir_valor_asignacion(value, source_type, target_type)**: aplica la conversión de valor específicamente en el contexto de una asignación, normalizando primero los *char* a su valor numérico entero cuando es necesario.
- **_valor_por_defecto(type_name, visited)**: genera el valor inicial de una variable recién declarada. Para tipos básicos consulta **DEFAULT_TYPES**; para registros construye recursivamente un diccionario con el valor por defecto de cada campo. El parámetro opcional *visited* evita recursión infinita en registros con referencias cíclicas.

3.4.3. Validación Semántica

Estos métodos encapsulan comprobaciones semánticas concretas que se invocan desde múltiples reglas de producción:

- **_condicion_es_valida(expr, line, contexto)**: verifica que la expresión usada como condición en un *if*, *while* o *do-while* sea de **tipo boolean**. De forma excepcional, también admite el tipo *unknown* para no interrumpir el análisis cuando el tipo de una expresión no ha podido resolverse completamente todavía. Si el tipo es incompatible, registra el error semántico con el número de línea y el nombre de la estructura de control afectada.
- **_registrar_firma_funcion(func_name, params, return_type, line)**: añade una nueva firma a la tabla de funciones, comprobando previamente que no exista ya una firma idéntica para el mismo nombre. Si la firma está duplicada, reporta un error semántico. Este mecanismo es el que permite la **sobrecarga de funciones**.

3.4.4. Exportación de Tablas Semánticas

Una vez completado el análisis, el parser puede volcar los resultados a disco mediante dos métodos:

- **_formatear_valor(value)**: convierte un valor Python al formato textual esperado en los ficheros de salida. Los booleanos se escriben como *true/false*, los registros como *{campo:valor,...}* y el resto como su representación en cadena directa.
- **exportar_tablas_semanticas(input_path)**: genera los tres ficheros de salida a partir del nombre del fichero de entrada analizado: *.symbols* con las variables globales, *.records* con los registros declarados y *.functions* con todas las firmas de funciones registradas. Cuando el

programa contiene funciones o estructuras de control (*_has_flow* o *self.functions*), la tabla de símbolos exporta únicamente el tipo de cada variable, omitiendo el valor.

3.4.5. Utilidades Específicas de cuartetos

- **_expr(type_name, value, ref=None)**: construye la representación (*tipo, valor, ref*). Si *ref* no se proporciona, intenta inferirlo a partir del valor mediante **_valor_a_operando**.
- **_extraer_expr(expr)**: descompone cualquier expresión (de longitud 2 o 3) en sus tres componentes (*tipo, valor, ref*), garantizando compatibilidad hacia atrás.
- **_nuevo_temporal** y **_nueva_etiqueta**, que generan nombres *@Tn* y *@Ln*.
- **_normalizar_operando_cuarteto(operand)**: convierte un operando al formato texto usado en *.quartets* (*_* para *None*, *true/false* para booleanos, y *str(operand)* para el resto).
- **_emit(operator, arg1, arg2, result)**: registra un cuarteto en el buffer *self.quartets*, normalizando todos sus campos.
- **_valor_a_operando(value, value_type=None)**: convierte valores semánticos concretos al formato de operando de cuarteto. Los *char* se serializan con comillas simples y con escape de ** y *'*.
- **_asegurar_tipo_en_cuartetos(ref, source_type, target_type)**: inserta cuartetos de conversión explícita de tipo cuando los operandos requieren promoción: *CHAR_TO_INT*, *INT_TO_FLOAT* o la cadena *CHAR_TO_INT + INT_TO_FLOAT* para *char* $\rightarrow$ *float*. Devuelve la referencia al temporal resultante.
- **_operador_binario_a_cuarteto(operator_type)**: mapea los tokens de operador binario de la gramática a sus opcodes de cuarteto (*PLUS* $\rightarrow$ *ADD*, *MINUS* $\rightarrow$ *SUB*, *TIMES* $\rightarrow$ *MUL*, *DIVIDE* $\rightarrow$ *DIV*, *GREATER* $\rightarrow$ *GT*, *GREATER_EQUAL* $\rightarrow$ *GTE*, *LESS* $\rightarrow$ *LT*, *LESS_EQUAL* $\rightarrow$ *LTE*, *EQUAL* $\rightarrow$ *EQ*, *AND* $\rightarrow$ *AND*, *OR* $\rightarrow$ *OR*).
- **_operador_unario_a_cuarteto(operator_type)**: mapea los tokens de operador unario a sus opcodes (*NOT* $\rightarrow$ *NOT*, *MINUS* $\rightarrow$ *UMINUS*, *PLUS* $\rightarrow$ *UPLUS*).
- **exportar_cuartetos(input_path)**: serializa *self.quartets* al fichero *.quartets*, escribiendo una línea por cuarteto con el formato *OPERADOR,ARG1,ARG2,RESULT*.

3.5. Comprobaciones Semánticas Implementadas

3.5.1. Declaraciones, Asignaciones y Sentencias Simples

Las funciones implicadas son **p_declaracion_variable**, **p_declaracion_o_funcion_tipada**, **p_asignacion** y **p_print_stmt**.

En las **declaraciones de variables**, el parser distingue entre declaraciones simples o múltiples sin inicialización, como *int a, b, c;*, y declaraciones con inicialización, como *int a = 5;*. En ambos casos se comprueba que el tipo declarado sea válido, es decir, que corresponda a un tipo básico del lenguaje o a un registro previamente definido. También se verifica que el identificador no haya sido declarado ya en el ámbito actual.

Cuando una variable se declara sin valor inicial, se registra con el valor por defecto de su tipo: *0* para **int**, *0.0* para **float**, *"* para **char** y *false* para **boolean**. En el caso de variables de tipo registro, el valor por defecto se construye recursivamente a partir de los valores por defecto de sus campos. La declaración múltiple con asignación, como *float a, b = 5;*, no se trata como caso semántico, ya que queda rechazada directamente por la gramática como error sintáctico.

En las asignaciones, **p_asignacion** valida que el lado izquierdo sea una variable o un acceso a campo válido. Después compara el tipo del destino con el tipo de la expresión asignada. Si ambos tipos coinciden, la asignación se acepta directamente; si existe una conversión implícita permitida, se convierte el valor antes de almacenarlo; y si los tipos son incompatibles, se reporta un error semántico.

La sentencia *print*, procesada en **p_print_stmt**, se trata como una función especial del sistema. Se comprueba que reciba al menos un argumento y que todos los argumentos sean expresiones válidas, descartando aquellos casos en los que algún argumento llegue marcado con tipo error.

3.5.2. Expresiones

3.5.2.1. Operaciones aritméticas

La función implicada es **p_expresion_binaria**, en la rama de los operadores **PLUS**, **MINUS**, **TIMES** y **DIVIDE**.

Los operadores **+**, **-**, ***** y **/** requieren que ambos operandos sean de **tipo numérico** (*char*, *int* o *float*). Si alguno de los operandos es de tipo *boolean* o de tipo *registro*, se reporta un error semántico. El tipo resultante se determina mediante **_tipo_numerico_comun**, que aplica la regla de promoción: *float* > *int* > *char*.

Existe un caso especial para ***** y **/**: si el tipo común resultaría ser *char*, se promociona a *int* para evitar desbordamientos.

Python

```
if operator_type in ('TIMES', 'DIVIDE') and result_type == 'char':  
    result_type = 'int'
```

Cuando ambos operandos tienen valor concreto en tiempo de análisis, el parser evalúa la operación directamente, propagando el resultado. La división entre cero produce *None* como valor, evitando excepciones en tiempo de compilación.

3.5.2.2. Operaciones Comparativas

La función implicada es **p_expresion_binaria**, en la rama de los operadores **GREATER**, **GREATER_EQUAL**, **LESS**, **LESS_EQUAL** y **EQUAL**.

Los operadores `>`, `>=`, `<` y `<=` exigen que ambos operandos sean numéricos, y devuelven siempre `boolean`. Para evaluarlos, ambos operandos se normalizan al mismo tipo numérico común mediante `_normalizar_operandos_numericos`.

El operador `==` es más permisivo: acepta tipos iguales o tipos compatibles por conversión en alguna de las dos direcciones. Cuando ambos operandos son numéricos, también se normalizan al mismo tipo numérico común antes de comparar; en los demás casos comparables, la igualdad se evalúa directamente sobre los valores.

3.5.2.3. Operaciones Lógicas

La función implicada es `p_expresion_binaria`, en la rama de los operadores **AND** y **OR**.

Los operadores `&&` (AND) y `//` (OR) requieren que ambos operandos sean estrictamente de **tipo boolean**. No se admite ninguna conversión automática: un `int` o `char` no pueden usarse directamente como booleanos. Si alguno de los operandos no es `boolean`, se reporta un error semántico. El resultado es siempre `boolean`.

3.5.2.4. Operadores Unarios

La función implicada es `p_expresion_unaria`.

El operador `!` (NOT) requiere un operando de **tipo boolean** y devuelve también `boolean`. Los operadores unarios `+` y `-` requieren un operando **numérico** (`char`, `int` o `float`). En el caso de `+`, el valor se propaga sin cambios; en el caso de `-`, se aplica la negación correspondiente al tipo del operando.

Internamente, el parser trabaja con expresiones tipadas que contienen al menos el tipo y el valor calculado, pudiendo incorporar información auxiliar adicional. A partir de esa representación, `p_expresion_unaria` extrae el tipo del operando y valida si el operador aplicado es compatible con él.

Si el operando tiene **tipo boolean**, solo se admite el operador `!`. Si el operando tiene **tipo numérico**, se admiten `+` y `-`. Cuando el operando es de **tipo char** y se aplica la negación unaria, esta se realiza mediante una inversión modular sobre el código ASCII: `chr((-ord(c)) % 256)`.

Si el operando tiene un **tipo no compatible** con el operador utilizado, se reporta el correspondiente error semántico y se devuelve una expresión marcada con tipo `error`, evitando así la propagación de falsos positivos en comprobaciones posteriores.

3.5.3. Registros

Las funciones principales implicadas son `p_declaracion_record`, `p_instanciacion` y `p_acceso`.

En la **declaración de registros**, el parser comprueba que no exista ya un registro con el mismo nombre, que todos los campos tengan tipos válidos y que no haya campos repetidos dentro del mismo registro. También se soporta la declaración abreviada de varios campos consecutivos con el mismo tipo, como en *record Point(float x1, x2);*, donde los campos sin tipo explícito heredan el último tipo indicado.

Durante la instanciación con **new**, se comprueba que el registro exista, que el número de argumentos coincida con el número de campos definidos y que cada argumento sea compatible con el tipo del campo correspondiente. Cuando procede, se aplican conversiones automáticas antes de construir la instancia interna del registro.

El **acceso a campos** mediante el operador punto se resuelve de forma encadenada, lo que permite expresiones como *linea.a.x1*. En cada nivel se valida que el tipo izquierdo sea un registro conocido y que el campo solicitado exista en su definición. Si el acceso es correcto, se conserva la ruta completa desde la variable raíz, lo que permite posteriormente actualizar campos anidados en asignaciones.

3.5.4. Funciones

3.5.4.1. Declaración y Sobrecarga

Las funciones implicadas son **p_declaracion_funcion_void**, **p_inicio_funcion_void** y **p_fin_funcion_void** para funciones *void*.

p_declaracion_o_funcion_tipada, **p_inicio_funcion_tipada** y **p_fin_funcion_tipada** para funciones con retorno tipado.

La validación de tipos de los parámetros se realiza en **p_parametro**.

La desambiguación entre declaración de variable y función en el ámbito global se gestiona mediante **p_marcar_declaracion_tipada** y **p_resto_tipado_programa**.

Al declarar una función, las reglas **p_inicio_funcion_void** y **p_inicio_funcion_tipada** comprueban que ningún parámetro tenga un tipo inválido. A continuación invocan **_registrar_firma_funcion**, que busca en *self.functions* si ya existe una entrada con el mismo nombre y la misma lista de tipos de parámetros: si es así, se emite un error semántico por firma duplicada; en caso contrario, se añade como una nueva sobrecarga válida.

Después de registrar la firma, se crea un nuevo ámbito local con los parámetros como entradas iniciales y se apila en *self.stack*. En este paso también se comprueba que no haya parámetros repetidos dentro de la misma función. Al salir de la función, **p_fin_funcion_void** y **p_fin_funcion_tipada** desapilan el ámbito local y limpian *self.current_function*.

El lenguaje Lava permite así varias funciones con el mismo nombre siempre que difieran en número o tipo de parámetros. La recursión directa no está permitida, pero esta restricción no se debe a que la firma no exista todavía, sino a una comprobación explícita realizada en **p_llamada**: si una función intenta invocarse a sí misma mientras su cuerpo se está analizando, se reporta un error semántico.

3.5.4.2. Invocación y Resolución de Sobrecarga

La función implicada es **p_llamada**.

Cuando se invoca una función, el parser realiza un proceso de resolución en varios pasos:

- **Se verifica que la función exista en la tabla de funciones** y que no se esté invocando desde su propio cuerpo (la recursión directa no está permitida).
- **Se buscan todas las firmas con el mismo nombre cuyo número de parámetros coincida con el número de argumentos proporcionados.**
- **Para cada firma candidata, se calcula el coste total de conversión** sumando el coste individual de cada argumento. Si un argumento no es convertible al tipo del parámetro correspondiente, esa firma se descarta.
- **Se da prioridad a la firma con coincidencia exacta** (coste 0). Si no existe, se selecciona la firma con menor coste total de conversión.
- **Si hay empate entre varias firmas** (ya sea por coincidencia exacta múltiple o por mismo coste mínimo), **se reporta un error de llamada ambigua**.

3.5.4.3. Validación de *return*

La función implicada es **p_sentencia_simple**, en la rama del token **RETURN**, **p_fin_funcion_tipada** y **p_declaracion_o_funcion_tipada**.

La regla **p_sentencia_simple** procesa las sentencias *return expr* en los contextos donde el lenguaje permite retorno. Cuando aparece un *return*, se comprueba en primer lugar que esté dentro de una función; si aparece fuera de cualquier función, se reporta un error semántico.

Si el *return* pertenece a una función con retorno tipado, se valida que el tipo de la expresión retornada sea compatible con el tipo declarado de la función, aplicando conversiones automáticas cuando sea necesario. Además, cada *return* correcto marca la función actual con el indicador *has_return*.

La comprobación de que una función tipada contenga al menos un *return* no se resuelve en la propia sentencia, sino al finalizar el análisis de la función: **p_fin_funcion_tipada** devuelve si se ha encontrado algún retorno y **p_declaracion_o_funcion_tipada** emite un error semántico si la función declarada exige valor de retorno y no se ha detectado ninguno.

En las funciones *void*, el uso de *return* con expresión no se trata aquí como un caso semántico adicional, ya que la gramática de los bloques *void* no admite ese tipo de sentencia.

3.5.5. Control de Flujo

Las funciones principales implicadas son `p_sentencia_if`, `p_sentencia_while`, `p_sentencia_do_while`, sus variantes para bloques `void`, `p_entrar_bucle`, `p_salir_bucle`, `p_sentencia_simple` y `p_sentencia_simple_void`.

Las estructuras *if*, *while* y *do-while* exigen que la condición sea de tipo *boolean*. No se admiten conversiones automáticas en este contexto, por lo que una expresión numérica como `5 / 7` no puede utilizarse como condición aunque su valor sea distinto de cero. Esta validación se centraliza en el método `_condicion_es_valida`, que recibe la expresión, la línea y el tipo de estructura de control analizada.

La sentencia *break* solo es válida dentro de un bucle. Para comprobarlo, el parser mantiene el contador `loop_depth`, que se incrementa al entrar en un *while* o *do-while* y se decrementa al salir. Si aparece un *break* cuando `loop_depth` es cero, se reporta un error semántico. Este mecanismo permite admitir correctamente *break* dentro de bucles anidados o dentro de bloques *if* contenidos en un bucle.

3.6. Gestión de Errores Semánticos

Cada vez que el parser detecta una violación semántica, activa el flag *has_semantic_error* e imprime un mensaje descriptivo por consola indicando el número de línea y la naturaleza del error. En lugar de detener inmediatamente la ejecución, el analizador intenta continuar el procesamiento del resto del programa siempre que la estructura sintáctica lo permita, devolviendo expresiones marcadas como *error* o finalizando únicamente la acción semántica de la regla afectada. De este modo, es posible acumular varios errores en una misma ejecución y ofrecer una salida de diagnóstico más completa.

Fragmento representativo basado en la función `p_declaracion_o_funcion_tipada`:

```
Python
self.has_semantic_error = True
print(f"[ERROR SEMANTICO] Linea {line}: No se puede asignar tipo '{expr_type}' "
      f"a '{id_name}' de tipo '{type_name}'")
return
```

El flag *has_semantic_error* permite que el módulo *main.py* determine el resultado final del proceso sin necesidad de capturar excepciones específicas del análisis: si al terminar el parseo se ha detectado algún error sintáctico o semántico, el programa finaliza con código de salida 1; en caso contrario, exporta las tablas semánticas y, en esta versión final, también el fichero de cuartetos asociado al código intermedio.

Adicionalmente, el tipo especial **unknown** se utiliza como mecanismo de tolerancia semántica en aquellos puntos donde no conviene rechazar de forma prematura una construcción cuyo tipo no ha podido resolverse completamente. Así, en lugar de propagar automáticamente un *error*, ciertas reglas permiten continuar el análisis con *unknown*, reduciendo la aparición de falsos positivos encadenados y haciendo el comportamiento del analizador más robusto frente a errores parciales previos.

3.7. Diseño Parte Opcional

La salida intermedia se escribe en un fichero con la misma base que el programa fuente y extensión *.quartets*. Cada línea contiene exactamente cuatro campos separados por comas: **OPERADOR,ARG1,ARG2,RESULT**

Cuando un operador no usa alguno de los campos, se escribe `_`. Los booleanos se serializan como *true* y *false*, los caracteres se representan entre comillas simples escapando los casos necesarios y los temporales y etiquetas siguen la convención *@T1*, *@T2*, ... y *@L1*, *@L2*, ...

La implementación genera cuartetos para las principales construcciones del lenguaje:

- asignaciones a variables simples (**ASSIGN**).
- operaciones binarias aritméticas (**ADD, SUB, MUL, DIV**).
- comparaciones (**GT, GTE, LT, LTE, EQ**).
- operaciones lógicas (**AND, OR**).
- operadores unarios (**NOT, UMINUS, UPLUS**).
- conversiones explícitas (**CHAR_TO_INT, INT_TO_FLOAT**).
- control de flujo (**JUMPF, JUMPT, JUMP, LABEL**).
- impresión (**PRINT**).
- invocaciones a función (**CALL**).

Por otro lado, para generar correctamente los saltos y etiquetas de las estructuras de control, se han introducido producciones vacías auxiliares, denominadas **marcadores**. Estas reglas no consumen tokens ni representan construcciones propias del lenguaje Lava; únicamente permiten ejecutar acciones de generación de código en puntos concretos del análisis LALR.

Estructura	Esquema de generación
<i>if</i> (<i>p_if_mark</i>)	Se evalúa la condición, se emite JUMPF hacia la etiqueta final y se coloca LABEL al terminar el bloque.
<i>if-else</i> (<i>p_else_mark</i> y <i>p_if_end_mark</i>)	Se emite JUMPF hacia la rama <i>else</i> , un JUMP al final tras el bloque <i>then</i> , y etiquetas para separar ambas ramas.
<i>while</i> (<i>p_while_start</i> y <i>p_while_end</i>)	Se coloca una etiqueta de inicio, se evalúa la condición, se emite JUMPF hacia la salida, se genera el cuerpo y se vuelve al inicio con JUMP .

do-while (p_do_start y p_do_end)	Se coloca una etiqueta al inicio del cuerpo, se genera el bloque, se evalúa la condición al final y se emite JUMPT para repetir si es verdadera.
---	---

Internamente, el parser mantiene contadores para crear temporales y etiquetas, además de pilas auxiliares para recordar el contexto activo de cada *if* o bucle anidado.

4. Batería de Pruebas

Se han diseñado nueve ficheros de entrada *.lava* que ejercen de forma exhaustiva todas las funcionalidades implementadas en el compilador: tipos primitivos, literales en múltiples notaciones, expresiones con conversiones automáticas, estructuras de control, registros simples y anidados, funciones tipadas y *void*, sobrecarga de funciones y casos límite del lexer. Todos los ficheros compilan sin errores, lo que garantiza que cada ejecución produce los cuatro ficheros de salida exigidos.

El directorio de pruebas sigue la siguiente **estructura**:

```
None
/pruebas
├─ /input      ← ficheros .lava de entrada
└─ /output
    ├─ /records ← ficheros .records de cada ejecución
    ├─ /functions ← ficheros .functions de cada ejecución
    ├─ /symbols  ← ficheros .symbols de cada ejecución
    └─ /quartets ← ficheros .quartets de cada ejecución
```

4.1. Tabla de cobertura

Fichero	Funcionalidades probadas	.records	.functions
01_tipos_y_literales.lava	4 tipos primitivos; enteros <i>dec/bin/oct/hex</i> ; <i>float</i> con punto y notación científica; <i>true/false</i> ; multi-declaración; declaración + inicialización; print de cada tipo	vacío	vacío
02_expresiones_y_conversiones.lava	+ - * / binarios; unarios - + !; comparativos; lógicos && !; conversiones <i>char→int</i> , <i>char→float</i> , <i>int→float</i> ; == permisivo; precedencia y paréntesis	vacío	vacío
03_control_flujo.lava	<i>if</i> simple; <i>if-else</i> ; <i>if</i> anidado; <i>while</i> ;	vacío	vacío

	<i>do-while</i> ; <i>break</i> en <i>while</i> y <i>do-while</i> ; condición compuesta		
04_registros.lava	<i>record</i> con tipos explícitos; herencia de tipo entre campos consecutivos; <i>new</i> ; acceso a campo; asignación a campo; <i>record</i> sin inicializar (valores por defecto); tipos mixtos	lleno	vacío
05_registros_anidados.lava	Registro con campo de tipo registro; acceso encadenado a.b.c; asignación a campo anidado; <i>new</i> anidado como argumento de otro <i>new</i>	lleno	vacío
06_funciones.lava	Funciones tipadas con <i>return</i> ; funciones <i>void</i> ; múltiples parámetros; variables locales; llamada como expresión y como sentencia; <i>print</i> dentro de función; función con <i>while</i> interno	vacío	lleno
07_sobrecarga.lava	Sobrecarga por tipo (<i>int</i> vs <i>float</i>); sobrecarga por aridad; resolución exacta; resolución por coste mínimo (<i>char</i> → <i>int</i> < <i>char</i> → <i>float</i>)	vacío	lleno
08_integracion.lava	Registros anidados + funciones con parámetro de tipo registro + <i>while/do-while</i> + <i>if-else</i> + sobrecarga	lleno	lleno
09_casos_limite.lava	Comentarios <i>//</i> y <i>/* */</i> multilínea; ; múltiples entre sentencias; paréntesis profundamente anidados; distintos valores char ASCII; <i>if</i> con condición compuesta	vacío	vacío

4.2. Análisis de resultados

4.2.1. Ficheros .symbols

Los tests **01**, **02**, **04**, **05** y **09** no contienen estructuras de control ni definiciones de función, por lo que el compilador exporta la tabla de símbolos en formato nombre: tipo,valor. Por ejemplo, **01_tipos_y_literales.symbols** refleja los cuatro enteros con valor 495 (todas las notaciones *dec/bin/oct/hex* encodifican el mismo número), los flotantes con su valor decimal final y las variables sin inicializar con el valor por defecto de su tipo (*0*, *0.0*, *"*, *false*).

Los tests **03, 06, 07 y 08** contienen control de flujo y/o funciones, por lo que los símbolos se exportan solo con tipo (nombre:tipo), ya que sus valores no son deterministas en tiempo de compilación.

4.2.2. Ficheros .records

Solo los tests que declaran *record* generan ficheros no vacíos:

- **04** muestra cuatro registros (Punto, Eje, Mezcla, Stats) con la herencia de tipo resuelta. Por ejemplo: *[x1:float,x2:float]* confirma que *x2* hereda el tipo *float* de *x1*.
- **05** muestra registros con campos de tipo registro: *Segmento:[inicio:Vec2,fin:Vec2]*, verificando que los tipos compuestos se serializan correctamente en la tabla.
- **08** combina registros anidados con funciones que los reciben como parámetro: *Rectangulo:[origen:Vec2,esquina:Vec2]*.

4.2.3. Ficheros .functions

Solo los tests con definiciones de función generan ficheros no vacíos:

- **06** registra las 7 firmas distintas con sus parámetros y tipo de retorno, incluyendo *factorial:[n:int],int*.
- **07** registra las 7 variantes de las tres funciones sobrecargadas (calcular, imprimir, combinar), validando que la tabla acumula todas las firmas por nombre. Por ejemplo: *calcular:[x:float],float* y *calcular:[x:int],int* coexisten.
- **08** valida que las dos sobrecargas de área coexisten: *area:[r:Rectangulo],float* y *area:[a:float,b:float],float*.

4.2.4. Ficheros .quartets

- **02** genera instrucciones **CHAR_TO_INT** e **INT_TO_FLOAT** para las conversiones automáticas, confirmando que el compilador emite las instrucciones de conversión explícitas en el código intermedio.
- **03** genera **JUMPF**, **JUMP** y **LABEL** para los *if/else* y *while*, y **JUMPT** con **LABEL** para el *do-while*, confirmando la generación correcta de saltos condicionales e incondicionales y el uso de etiquetas *@L#*.
- **06** genera instrucciones **CALL** para cada invocación de función, validando la interacción entre el análisis semántico y la generación de código intermedio.
- **07** valida la resolución de sobrecarga a nivel de cuartetos: la llamada *calcular('A')* produce **CHAR_TO_INT** pero no **INT_TO_FLOAT**, confirmando que se eligió la versión *int* (coste 1) frente a la *float* (coste 2).
- **08** combina todas las instrucciones anteriores en un programa realista: **ASSIGN**, **ADD**, **MUL**, **JUMPF**, **LABEL**, **JUMP**, **JUMPT**, **CALL** y conversiones de tipo aparecen en el mismo fichero, ejerciendo la interacción entre todas las features.
- **09** confirma que las instrucciones **PRINT** se emiten también en presencia de expresiones con paréntesis profundamente anidados, validando que la precedencia no rompe la generación de cuartetos.

Los nueve tests demuestran que el **compilador procesa correctamente todas las especificaciones del lenguaje Lava**.

5. Conclusiones

La práctica se ha completado cumpliendo los objetivos previstos de la tercera entrega: se ha implementado un **analizador semántico** integrado en el parser de Lava, capaz de detectar y reportar errores de tipo, redeclaración de variables, uso incorrecto de sentencias de control y validación de firmas de funciones. Además, se ha desarrollado la **parte opcional de generación de código intermedio**, incorporando la emisión de cuartetos dentro del propio analizador sintáctico-semántico.

Entre las **decisiones de diseño adoptadas**, destaca la **gestión de la pila de ámbitos** mediante una lista de diccionarios apilables, lo que permite modelar correctamente el alcance léxico del lenguaje y separar de forma natural variables globales y locales. También resulta especialmente relevante la **ampliación de la representación interna de las expresiones** a la forma (**tipo, valor, ref**), ya que permite combinar en una misma estructura la información necesaria para el análisis semántico y para la generación de cuartetos.

En cuanto a la **resolución de sobrecarga**, la selección de firma basada en el coste mínimo de conversión permite un comportamiento coherente con el enunciado: se priorizan siempre las coincidencias exactas y solo en su ausencia se consideran conversiones automáticas compatibles.

La validación se ha realizado mediante una **batería de pruebas** de nueve ficheros .lava, detallada en la sección correspondiente, que cubre de forma sistemática todos los tipos primitivos, los operadores y sus conversiones automáticas, las estructuras de control, los registros simples y anidados, las funciones con y sin sobrecarga, y casos límite del lexer.

En conjunto, el compilador de Lava queda en un estado final completo y coherente: no sólo verifica la corrección semántica de los programas, sino que además genera una representación intermedia en forma de cuartetos, lo que refuerza el carácter integral de la solución desarrollada.

ANEXO. Declaración de Uso de Inteligencia Artificial

Durante el desarrollo de esta práctica, se emplearon herramientas de IA generativa exclusivamente como apoyo técnico. El diseño, la implementación del código fuente y la toma de decisiones recaen íntegramente en los estudiantes.

Uso de Perplexity AI

Perplexity AI se empleó como herramienta de consulta y apoyo en los siguientes ámbitos:

- **Diseño de acciones semánticas:** consulta de estrategias para integrar la lógica semántica dentro de las reglas de producción de PLY sin introducir conflictos LALR ni efectos secundarios indeseados.

- **Gestión de ámbitos y pila de contextos:** apoyo para definir el modelo de pila de ámbitos más adecuado para el alcance léxico del lenguaje Lava, incluyendo la creación y destrucción de ámbitos en la entrada y salida de funciones.
- **Resolución de sobrecarga de funciones:** consulta sobre estrategias de resolución de sobrecarga basadas en coste de conversión, para determinar qué firma seleccionar cuando múltiples candidatas son compatibles con los argumentos de una llamada.
- **Apoyo en la batería de pruebas:** ayuda en la elaboración de casos de prueba variados que cubrieran los casos límite adecuados.
- **Generación de código intermedio (parte opcional):** consulta y apoyo para comprender qué es el código intermedio y cómo se programa en la práctica. Se usó como guía de aprendizaje para entender el modelo de cuartetos, el funcionamiento de temporales y etiquetas, y cómo integrar la generación de instrucciones dentro de las reglas de producción de PLY, resolviendo dudas conceptuales.

Uso de Claude Opus 4.6

Se empleó para la revisión del código, detección de inconsistencias y como apoyo directo en la implementación de funcionalidades avanzadas:

- **Corrección en `_convertir_numero`:** se añadió una comprobación previa para capturar el caso de char vacío (") antes de invocar `ord()`, evitando un error en tiempo de ejecución por longitud inválida.
- **Herencia de tipo en campos de registro:** se habilitó la herencia de tipo entre campos consecutivos de un registro (por ejemplo, `record Point(float x1, x2)`), de modo que `campo_record_item` acepta tanto tipo `ID` como `ID` solo, y `p_declaracion_record` resuelve los tipos `None` con el último tipo explícito visto en la lista.
- **Implementación de la parte opcional (Generación de código intermedio):** la IA actuó como guía durante la implementación de esta fase, acompañando el proceso de forma progresiva. A través de una serie de consultas iterativas, ayudó a decidir cómo estructurar la emisión de cuartetos dentro de las reglas de producción existentes, cómo gestionar los contadores de temporales y etiquetas, y cómo razonar los saltos de control de flujo en estructuras como `if-else`, `while` y `do-while`.

Verificación y Responsabilidad

Las respuestas de ambas herramientas se utilizaron únicamente como referencia. Toda sugerencia fue sometida a revisión crítica, análisis frente a las especificaciones del lenguaje Lava y validación funcional. El uso de IA no ha sustituido el proceso de aprendizaje, y los estudiantes pueden explicar y defender todas las decisiones de código adoptadas.